

Opta Modbus Test Sketch

A test sketch for the **Arduino Opta** that exercises both Modbus TCP and Modbus RTU (selectable via a `#define`). The sketch acts as a **Modbus client** (master): it reads registers from a server, writes an incrementing counter, and polls a command register — echoing the command back and writing a doubled value.

Hardware Requirements

- [Arduino Opta](#) (any variant with Ethernet)
 - Ethernet cable connected to your laptop or local network switch
 - For RTU mode: RS-485 wiring to a Modbus RTU server device
-

Software Requirements

Arduino IDE

Download and install the [Arduino IDE 2.x](#) if you haven't already.

Board Support Package

The Opta requires the **Arduino Mbed OS Opta Boards** package.

1. Open Arduino IDE → **File** → **Preferences**
2. Add the following URL to *Additional boards manager URLs*:

None

https://downloads.arduino.cc/packages/package_index.json

3. Go to **Tools** → **Board** → **Boards Manager**
4. Search for **Arduino Mbed OS Opta Boards** and click **Install**
5. Select **Tools** → **Board** → **Arduino Mbed OS Opta Boards** → **Arduino Opta**

Required Libraries

Install all three libraries via **Tools** → **Manage Libraries** in the Arduino IDE.

Library	Purpose	Search term
ArduinoModbus	Modbus TCP & RTU client/server	ArduinoModbus
ArduinoRS485	RS-485 physical layer (dependency of ArduinoModbus)	ArduinoRS485
Ethernet	Ethernet/TCP networking for the Opta	Ethernet

Note: ArduinoRS485 is a dependency of ArduinoModbus. The Library Manager may install it automatically; if not, install it manually.

Network Configuration

Edit the configuration block near the top of the sketch to match your network:

```
C/C++
byte mac[]      = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
IPAddress ip     (172, 16, 0, 2);    // Opta static IP
IPAddress server (172, 16, 0, 1);    // IP of the Modbus TCP server
                                   (your laptop)
IPAddress gateway (172, 16, 0, 1);
IPAddress subnet (255, 255, 255, 0);
const int MODBUS_TCP_PORT = 502;
```

The Opta is assigned a **static IP**. Make sure your laptop (the Modbus server) is reachable at the **server** address.

Modbus Server Setup (Laptop Side)

This sketch acts as the **client**. You need a Modbus TCP server running on your laptop. [ModbusPal](#) works well for testing.

Before running the sketch, pre-configure the server with the following registers:

Register type	Addresses	Pre-set value	Purpose
Input Register	1	1234	Device ID (read once at startup)
Holding Register	0	—	Counter written by Opta
Holding Register	1	(write any value)	Command for Opta to read
Holding Register	2	—	Command echo written by Opta
Holding Register	3	—	Command × 2 written by Opta

Selecting TCP vs RTU Mode

At the top of the sketch, **comment out one** of the two defines:

```
C/C++
#define USE_MODBUS_TCP    // ← TCP mode (default)
// #define USE_MODBUS_RTU
```

To switch to RTU, flip them:

```
C/C++
// #define USE_MODBUS_TCP
#define USE_MODBUS_RTU    // ← RTU mode
```

Behavior at Runtime

1. **Startup** — Waits for Ethernet link, connects to the Modbus TCP server, reads the Device ID from Input Register 1, then pauses 10 seconds so you can read the Serial output.
2. **Counter** — Every 1 second, writes an incrementing counter (1–10, wrapping) to Holding Register 0.
3. **Command poll** — Every 500 ms, reads Holding Register 1. If a command is present, it:
 - Echoes the value to Holding Register 2
 - Writes **command × 2** to Holding Register 3

4. **Reconnection** — If the TCP connection drops, the sketch automatically attempts to reconnect on the next loop iteration.

Serial Monitor

Open **Tools** → **Serial Monitor** at **9600 baud** to observe runtime output:

```
None
Opta Modbus Test Starting...
Waiting for Ethernet link. Link up.
Opta IP: 172.16.0.2
Device ID read from IR 1: 1234
Counter: 1
Counter: 2
Command received: 7
...
```

Troubleshooting

Symptom	Likely cause
Stuck on <code>Waiting for Ethernet link</code>	Cable unplugged or switch not ready
<code>Failed to connect to Modbus TCP server</code>	Server IP wrong, server not running, or firewall blocking port 502
<code>HR read error</code>	Register address out of range on the server, or connection dropped mid-transaction
Device ID reads as <code>-1</code>	Input Register 1 not pre-configured on the server
Sketch compiles but won't upload	Wrong board selected — confirm Arduino Opta in Tools → Board

Opta MQTT Test Sketch

A test sketch for the **Arduino Opta WiFi** that exercises MQTT over WiFi. The sketch acts as an **MQTT client**: it connects to a broker running on your laptop, announces its presence, publishes a heartbeat counter every second, and responds to incoming commands by echoing them and publishing a doubled result. A Last Will message is configured so the broker automatically marks the device offline if it disconnects unexpectedly.

Hardware Requirements

- [Arduino Opta WiFi](#) (the WiFi variant)
 - 2.4 GHz WiFi network accessible by both the Opta and your laptop
-

Software Requirements

Arduino IDE

Download and install the [Arduino IDE 2.x](#) if you haven't already.

Board Support Package

1. Open Arduino IDE → **File** → **Preferences**
2. Add the following URL to *Additional boards manager URLs*:

None

https://downloads.arduino.cc/packages/package_index.json

3. Go to **Tools** → **Board** → **Boards Manager**
4. Search for **Arduino Mbed OS Opta Boards** and click **Install**
5. Select **Tools** → **Board** → **Arduino Mbed OS Opta Boards** → **Arduino Opta**

Required Libraries

Install both libraries via **Tools** → **Manage Libraries**.

Library

Purpose

Search term

WiFi (Arduino Mbed Opta)	WiFi connectivity	(bundled with Opta board package — no separate install needed)
ArduinoMqttClient	MQTT client	<code>ArduinoMqttClient</code>

Note: The `WiFi.h` used here is the one bundled with the Mbed OS Opta board package, not the older AVR WiFi library. As long as you have the correct board package installed, no separate WiFi library install is required.

Laptop-Side Broker Setup (Mosquitto)

This sketch requires an **MQTT broker** running on your laptop. [Mosquitto](#) is a free, lightweight broker that works well for this purpose.

1. Install Mosquitto

- **Windows:** Download the installer from [mosquitto.org/download](#) and run it. Make note of the install directory (typically `C:\Program Files\mosquitto\`).
- **macOS:** `brew install mosquitto`
- **Linux (Debian/Ubuntu):** `sudo apt install mosquitto mosquitto-clients`

2. Configure Mosquitto

By default, Mosquitto 2.x does **not** allow connections from outside `localhost`. You need to create a configuration file to allow the Opta to connect.

Create a file called `mosquitto_test.conf` anywhere convenient (e.g., your Desktop) with the following contents:

```
None
listener 1883
allow_anonymous true
```

This opens port 1883 and disables authentication — fine for local testing. Do not use this configuration on a public or production network.

3. Start Mosquitto with the Config File

Open a terminal (Command Prompt or PowerShell on Windows) and run:

Windows:

None

```
"C:\Program Files\mosquitto\mosquitto.exe" -v -c  
C:\path\to\mosquitto_test.conf
```

macOS / Linux:

Shell

```
mosquitto -v -c /path/to/mosquitto_test.conf
```

The `-v` flag enables verbose logging so you can see connections and messages in the terminal. Leave this window open while testing.

4. Find Your Laptop's IP Address

The Opta needs to reach the broker. Find your laptop's local IP:

- **Windows:** Run `ipconfig` in a terminal and look for the IPv4 address on your WiFi adapter.
- **macOS/Linux:** Run `ifconfig` or `ip addr` and look for the address on your WiFi interface.

This is the address you will enter as `BROKER_IP` in the sketch (see below).

5. Allow Port 1883 Through Your Firewall

If the Opta cannot reach the broker, your laptop's firewall may be blocking port 1883.

- **Windows:** Go to **Windows Defender Firewall** → **Advanced Settings** → **Inbound Rules** → **New Rule**. Create a rule for port 1883, TCP, Allow.
- **macOS:** Go to **System Settings** → **Network** → **Firewall** → **Options** and ensure Mosquitto is allowed, or temporarily disable the firewall for testing.

Sketch Configuration

Edit the configuration block near the top of the sketch:

C/C++

```
const char* WIFI_SSID      = "YourNetworkName";
const char* WIFI_PASSWORD = "YourPassword";

IPAddress localIP (10, 0, 0, 101);  // Desired static IP for the
Opta
IPAddress gateway (10, 0, 0, 1);    // Your router's IP
IPAddress subnet  (255, 255, 255, 0);
IPAddress dns     (8, 8, 8, 8);

const char* BROKER_IP      = "10.0.0.100";  // Your laptop's IP
const int   BROKER_PORT    = 1883;
```

Static IP note: The Opta requests a static IP via `WiFi.config()`. If the IP you request is outside your router's DHCP pool, this usually works without conflicts. If you see the IP mismatch warning in the Serial Monitor, update `localIP` to the address the router actually assigned.

MQTT Topic Map

Topic	Direction	Retained	Purpose
<code>opta/status</code>	Opta → Broker	✓	"online" on connect; "offline" via Last Will on disconnect
<code>opta/device-id</code>	Opta → Broker	✓	Device ID published once on connect (1234)
<code>opta/heartbeat</code>	Opta → Broker	✗	Incrementing counter (1–10) published every second
<code>opta/commands/input</code>	Broker → Opta	✗	Send an integer command to the Opta from here
<code>opta/commands/echo</code>	Opta → Broker	✗	Opta echoes the received command value

opta/commands
/result

Opta →
Broker



Opta publishes command × 2

Testing with MQTT Explorer or mosquitto_pub / mosquitto_sub

Once the broker is running and the Opta is connected, you can interact with it from a second terminal window on your laptop.

Subscribe to all Opta topics:

Shell

```
mosquitto_sub -h localhost -t "opta/#" -v
```

Send a command to the Opta:

Shell

```
mosquitto_pub -h localhost -t "opta/commands/input" -m "7"
```

You should see the Opta respond with **7** on `opta/commands/echo` and **14** on `opta/commands/result`.

Alternatively, [MQTT Explorer](#) is a free GUI tool that makes it easy to browse all topics and publish test messages.

Behavior at Runtime

1. **Startup** — Configures static IP, connects to WiFi, then connects to the MQTT broker.
2. **On MQTT connect** — Publishes `"online"` (retained) to `opta/status`, publishes device ID `1234` (retained) to `opta/device-id`, and subscribes to `opta/commands/input`.
3. **Heartbeat loop** — Every 1 second, publishes an incrementing counter (1–10, wrapping) to `opta/heartbeat`.

4. **Command handling** — When a message arrives on `opta/commands/input`, the Opta echoes it to `opta/commands/echo` and publishes the doubled value to `opta/commands/result`.
 5. **Last Will** — If the Opta drops off the network unexpectedly, the broker automatically publishes `"offline"` (retained) to `opta/status`.
 6. **Reconnection** — If WiFi or MQTT connectivity is lost, the sketch halts after 10 failed attempts and prompts you to reset the Opta.
-

Serial Monitor

Open **Tools** → **Serial Monitor** at **115200 baud** to observe runtime output:

```
None
Program has begun.
Connecting to WiFi....
WiFi connected to: YourNetworkName
Static IP configured successfully. IP: 10.0.0.101
Connecting to MQTT broker.
MQTT connected.
Announced presence by setting presence to online
Device ID published: 1234
Subscribed to opta/commands/input
Heartbeat: 1
Heartbeat: 2
Command received: 7
Echo: 7  Result: 14
```

Troubleshooting

Symptom	Likely cause
Stuck on <code>Connecting to WiFi</code>	Wrong SSID/password, or Opta out of range
<code>WL_NO_SSID_AVAIL</code>	Network name not found — check for typos, or ensure it's a 2.4 GHz network

Connection refused
(MQTT error -2)

Broker not running, or `listener` line missing from config

Connection timeout
(MQTT error -1)

Wrong `BROKER_IP`, or firewall blocking port 1883

IP mismatch warning

Router assigned a different IP — update `localIP` to match, or reserve the IP in your router's DHCP settings

No messages seen on
`opta/heartbeat`

Broker running but not reachable — confirm laptop and Opta are on the same subnet

Sketch compiles but won't upload

Wrong board selected — confirm **Arduino Opta** in Tools → Board

Opta OpenADR 2.0b VEN Test

Three files implement a complete **OpenADR 2.0b** test between an Arduino Opta WiFi (VEN) and a Python server running on your laptop (VTN).

File	Runs on	Purpose
<code>upload_template.ino</code>	Opta	One-time utility: writes XML templates to Opta flash
<code>opta_ven.ino</code>	Opta	VEN: registers, polls, handles events, sends reports
<code>vtn_test_server.py</code>	Laptop	VTN: accepts registration, sends SIMPLE events, validates reports

Run order: `vtn_test_server.py` first, then `upload_template.ino` (once), then `opta_ven.ino`.

Hardware Requirements

- [Arduino Opta WiFi](#) (WiFi variant required)
 - 2.4 GHz WiFi network accessible by both the Opta and your laptop
-

Software Requirements

Arduino IDE

Download and install the [Arduino IDE 2.x](#) if you haven't already.

Board Support Package

1. Open Arduino IDE → **File** → **Preferences**
2. Add the following URL to *Additional boards manager URLs*:

None

https://downloads.arduino.cc/packages/package_index.json

3. Go to **Tools** → **Board** → **Boards Manager**
4. Search for **Arduino Mbed OS Opta Boards** and click **Install**
5. Select **Tools** → **Board** → **Arduino Mbed OS Opta Boards** → **Arduino Opta**

Arduino Libraries

Install via **Tools** → **Manage Libraries**.

Library	Purpose	Search term	Used by
ArduinoHttpClient	HTTP POST/GET to VTN	ArduinoHttpClient	opta_ven.ino
WiFi (Mbed Opta)	WiFi connectivity	<i>(bundled with board package)</i>	Both sketches

Note: [BlockDevice](#), [LittleFileSystem](#), and [MBRBlockDevice](#) are part of the Mbed OS core bundled with the Opta board package. No separate install is needed.

Python Dependencies ([vtn_test_server.py](#))

Requires Python 3.8+.

Important: openleadr is no longer under active maintenance. The current PyPI release (0.5.35) is the last published version, and its dependency chain — specifically [signxml-openadr](#) — requires older, pinned versions of [cryptography](#) and [pyOpenSSL](#) that conflict with what a plain `pip install openleadr` will attempt to resolve on modern systems. Installing without version pins will produce dependency conflicts or a broken environment. Use the exact versions below.

Step 1 — Install the pinned cryptography stack first

[signxml-openadr](#) (pulled in automatically by openleadr) requires [cryptography](#) `< 4` and [pyOpenSSL](#) `< 20`. Install these before openleadr so pip does not pull in newer incompatible versions:

Shell

```
pip install "cryptography==3.4.8" "pyOpenSSL==19.1.0"
```

Step 2 — Install openleadr and remaining dependencies

Shell

```
pip install "openleadr==0.5.1" "nest_asyncio" "aiohttp"
```

pip will resolve and install the remaining dependencies (`signxml-openadr`, `apscheduler`, `xmltodict`, `lxml`, and others) automatically within the constraints set in Step 1.

Verification

Confirm the install resolved correctly:

Shell

```
pip show openleadr cryptography pyOpenSSL
```

Expected output (versions must match):

None

```
Name: openleadr
```

```
Version: 0.5.1
```

```
...
```

```
Name: cryptography
```

```
Version: 3.4.8
```

```
...
```

```
Name: pyOpenSSL
```

```
Version: 19.1.0
```

If any version differs, re-run Steps 1 and 2. Using a virtual environment (below) is the most reliable way to avoid conflicts with other installed packages.

Recommended: use a virtual environment

Installing into a virtual environment keeps these older packages isolated from the rest of your Python installation:

Shell

```
python -m venv openleadr-env
```

```
# Activate (Windows)
```

```
openleadr-env\Scripts\activate
```

```
# Activate (macOS/Linux)
```

```
source openleadr-env/bin/activate
```

```
# Then install
```

```
pip install "cryptography==3.4.8" "pyOpenSSL==19.1.0"
```

```
pip install "openleadr==0.5.1" "nest_asyncio" "aiohttp"
```

Remember to activate the environment (`openleadr-env\Scripts\activate` or `source openleadr-env/bin/activate`) each time before running `vtn_test_server.py`.

Full dependency reference

The complete set of packages installed by the above commands:

Package	Version	Role
openleadr	0.5.1	OpenADR 2.0b server framework
nest_asyncio	latest	Allows nested event loops
aiohttp	latest	HTTP server backend for openleadr
signxml-openadr	2.9.x	XML signing (pulled in by openleadr)
cryptography	3.4.8	Cryptographic primitives — must be pinned
pyOpenSSL	19.1.0	TLS support — must be pinned
lxml	4.9.x	XML parsing (pulled in by signxml-openadr)
apscheduler	latest	Job scheduling (pulled in by openleadr)
xmltodict	latest	XML/dict conversion (pulled in by openleadr)

Network Configuration

Both sketches and the VTN server must be on the same subnet. The defaults used in all three files are:

Device	IP address	Port
Opta (VEN)	10.0.0.101	—
Laptop (VTN)	10.0.0.100	8080
Opta upload utility	10.0.0.101	8888

Edit the configuration block in each sketch/script to match your actual network. In both `.ino` files:

```
C/C++
const char* WIFI_SSID      = "YourNetworkName";
const char* WIFI_PASSWORD = "YourPassword";

IPAddress localIP  (10, 0, 0, 101);  // Desired static IP for the
Opta
IPAddress gateway  (10, 0, 0,  1);  // Your router's IP
IPAddress subnet   (255, 255, 255, 0);
IPAddress dns       (8, 8, 8,  8);
```

In `opta_ven.ino`, also update the VTN address:

```
C/C++
const char* VTN_HOST = "10.0.0.100";  // Your laptop's IP
const int   VTN_PORT = 8080;
```

In `vtn_test_server.py`:

```
Python
VTN_ID      = "test-vtn"
```



```
EXPECTED_VEN = "ven1234"
HOST          = "10.0.0.100"    # Your laptop's IP
PORT          = 8080
```

Flash Filesystem — Important Notes

The Opta's internal flash is partitioned into two regions:

Partition	Contents	Rule
1 (first 1 MB)	WiFi firmware	Never reformat or overwrite
2 (remainder)	User data (XML templates)	Safe to format and write

Both sketches enforce this by mounting only partition 2 (`usr_bd`). If the user partition is not found it is formatted automatically. Partition 1 is never touched.

Step 1 — Upload the XML Template (`upload_template.ino`)

This sketch runs a small HTTP server on the Opta that accepts the XML template file over a `curl` POST, writes it to flash, and confirms success. **Run this once before deploying the VEN sketch.**

1a. Flash the Upload Utility

Open `upload_template.ino` in Arduino IDE, update the WiFi credentials and IP, then upload to the Opta.

1b. Open Serial Monitor at 9600 baud

Confirm the Opta connects to WiFi and prints:

```
None
HTTP server started on port 8888
```

1c. Upload the Template File

From a terminal on your laptop, navigate to the directory containing `openadr_template.xml` and run:

```
Shell
curl -X POST http://10.0.0.101:8888 \
  -H "X-Filename: openadr_template.xml" \
  --data-binary @openadr_template.xml
```

A successful upload prints in both the terminal and Serial Monitor:

```
None
Upload OK: /fs/openadr_template.xml
Written : XXXX bytes
Verified: XXXX bytes on flash
```

1d. Verify the Upload (Optional)

Read the file back from flash to confirm it was written correctly:

```
Shell
curl http://10.0.0.101:8888/openadr_template.xml
```

The full XML content should print in your terminal.

Once confirmed, you do not need to run this sketch again unless you want to update the template file.

Step 2 — Start the VTN (`vtn_test_server.py`)

The VTN must be running **before** the Opta boots, as the Opta attempts to register immediately on startup.

2a. Start the Server

Shell

```
python vtn_test_server.py
```

You should see:

None

```
VTN starting
```

```
  VTN ID : test-vtn
```

```
  Host   : 10.0.0.100:8080
```

```
  Opta IP: 10.0.0.101
```

```
Server started.
```

```
Console thread started.
```

```
Queue processor started.
```

```
Console thread started. Waiting for Opta to register...
```

The VTN is now listening on port 8080. Leave this terminal open for the duration of the test.

If the server fails to bind, check that nothing else is using port 8080 and that your firewall allows inbound TCP connections on that port.

VTN Architecture Notes

The server runs three concurrent components:

Component	Type	Purpose
openleadr HTTP server	asyncio coroutine	Handles all OpenADR HTTP endpoints
Console thread	OS thread	Waits for Enter keypresses to trigger events without blocking the event loop
Event queue processor	asyncio task	Bridges the console thread to the async event loop via a thread-safe asyncio.Queue

This architecture is necessary because **input()** is a blocking call and cannot run directly inside an asyncio event loop.

Step 3 — Run the VEN (**opta_ven.ino**)

3a. Flash the VEN Sketch

Open **opta_ven.ino** in Arduino IDE, confirm all configuration matches your network and VTN, then upload to the Opta.

3b. Monitor Serial Output at 9600 baud

Expected startup sequence:

None

```
Opta OpenADR VEN starting...
Initializing block device... OK
Setting up partition table... OK
Mounting filesystem... OK
Loading template from /fs/openadr_template.xml
Template loaded. Size: XXXX bytes
Templates parsed into RAM sections:
  oadrCreatePartyRegistration : XXX bytes
  oadrRegisterReport          : XXX bytes
  oadrPoll                    : XXX bytes
  oadrCreatedEvent            : XXX bytes
  oadrUpdateReport            : XXX bytes
All templates loaded successfully.
Connecting to WiFi... connected.
Static IP configured successfully. IP: 10.0.0.101

--- Party Registration ---
[request/response XML logged here]
VEN registered. Device ID: ven1234

>>> Take screenshot now. Polling starts in 30 seconds...
>>> Starting poll loop.
Poll acknowledged - no pending events.
Poll acknowledged - no pending events.
...
```

After registration the sketch pauses 30 seconds before beginning the poll loop. This is intentional and gives you time to capture the registration exchange.

Simultaneously, the VTN terminal will print:

```
None
Registration request from VEN: ven1234
VEN ven1234 recognised. Registration confirmed.

Opta registered. Ready for test sequence.
Press Enter to send SIMPLE level 1 (moderate shed)...
```

Step 4 — Run the Test Sequence

The VTN console thread drives the test interactively. Both the VTN terminal and the Opta Serial Monitor should be visible at the same time.

Event 1 — Level 1 (Light Shed)

Press **Enter** in the VTN terminal. The VTN queues a SIMPLE signal level 1 event. On its next poll the Opta detects the event and:

1. Acknowledges with `oadrCreatedEvent` (optIn)
2. Reports 3 kW shed via `oadrUpdateReport`

VTN terminal output:

```
None
Queueing level 1 event...
Event queued.
Waiting for Opta compliance report...
  Report received after X.X seconds.
  Reported value : 3 kW
  Expected value : 3 kW - PASS

>>> Screenshot opportunity: level 1 report received.
>>> Press Enter when ready to continue...
```

Opta Serial Monitor output:

None

```
Event detected in response.  
Event ID: [event-id]  
Parsed signal level: 1  
Signal 1: Light shed. Reporting 3 kW.  
Acknowledging event: [event-id]  
Sending report: shed = 3 kW
```

Event 2 — Level 2 (Moderate Shed)

Press **Enter** again. The VTN sends a SIMPLE signal level 2 event. The Opta reports 10 kW shed.

VTN terminal output:

None

```
Reported value : 10 kW  
Expected value : 10 kW - PASS  
  
>>> Screenshot opportunity: level 2 report received.  
>>> Press Enter when ready to continue...  
  
Test sequence complete. Press Ctrl+C to stop the server.
```

OpenADR Event Flow

Step	Message	Direction	OpenADR endpoint
1	<code>oadrCreatePartyRegistration</code>	VEN → VTN	<code>/EiRegisterParty</code>
2	<code>oadrCreatedPartyRegistration</code>	VTN → VEN	<i>(response)</i>
3	<code>oadrPoll</code>	VEN → VTN	<code>/OadrPoll</code>
4	<code>oadrDistributeEvent</code>	VTN → VEN	<i>(poll response)</i>

5	<code>oadrCreatedEvent</code> (optIn)	VEN → VTN	<code>/EiEvent</code>
6	<code>oadrUpdateReport</code>	VEN → VTN	<code>/EiReport</code>

SIMPLE Signal Levels

Signal level	Meaning	Reported shed
0	Normal operations	0 kW
1	Light shed	3 kW
2	Moderate shed	10 kW
3	Emergency shed	15 kW

XML Template Structure

The file `openadr_template.xml` must contain the following five named XML blocks. Each is extracted at boot into a separate RAM string; all token substitutions are performed in RAM with no further flash reads.

Template block	OpenADR message	Tokens
<code>oadr:oadrCreatePartyRegistration</code>	Registration request	<code>{{VEN_ID}}</code> , <code>{{VTN_ID}}</code> , <code>{{REQUEST_ID}}</code>
<code>oadr:oadrRegisterReport</code>	Report registration	<code>{{VEN_ID}}</code> , <code>{{VTN_ID}}</code> , <code>{{REQUEST_ID}}</code>
<code>oadr:oadrPoll</code>	Poll request	<code>{{VEN_ID}}</code> , <code>{{VTN_ID}}</code> , <code>{{REQUEST_ID}}</code>
<code>oadr:oadrCreatedEvent</code>	Event acknowledgement	<code>{{VEN_ID}}</code> , <code>{{EVENT_ID}}</code> , <code>{{OPT_TYPE}}</code> , <code>{{REQUEST_ID}}</code>
<code>oadr:oadrUpdateReport</code>	Compliance report	<code>{{VEN_ID}}</code> , <code>{{REPORT_ID}}</code> , <code>{{REPORT_VALUE}}</code>

All messages are wrapped in the standard `oadrPayload` / `oadrSignedObject` envelope with the `http://openadr.org/oadr-2.0b/2012/07` namespace before being POSTed.

Retry and Failure Behavior

Condition	Behavior
WiFi fails after 10 attempts	Halts — reset Opta to retry
HTTP POST fails	Retries up to 3 times with 1-second delay
Poll fails 3 consecutive times	Halts — reset Opta to retry
Template file missing at boot	Halts with message to run upload utility
Critical template section empty	Halts with message to check and re-upload
VTN registration times out (>5 min)	Console thread exits — restart VTN and Opta

Troubleshooting

Symptom	Likely cause
<code>ERROR: template file not found</code>	Upload utility not run yet, or upload failed — re-run Step 1
<code>ERROR: one or more critical templates missing</code>	Template XML malformed or missing a required block — check tag names match the table above
<code>FAILED to format user partition</code>	Flash hardware issue — try power-cycling the Opta
Registration response empty on Opta	VTN not running, wrong <code>VTN_HOST/VTN_PORT</code> , or firewall blocking port 8080
VTN prints <code>Unexpected VEN name</code>	VEN ID mismatch — confirm <code>VEN_ID</code> in <code>opta_ven.ino</code> matches <code>EXPECTED_VEN</code> in the Python script (both should be <code>ven1234</code>)

Stuck on `Connecting to WiFi`

Wrong SSID/password, or 5 GHz-only network (Opta requires 2.4 GHz)

IP mismatch warning on Opta

Router assigned a different IP — update `localIP` or reserve the IP in your router's DHCP settings

No compliance report received (VTN times out)

Check Opta Serial Monitor — did it receive the event? Did it attempt to send a report? Watch VTN log for incoming EiReport POST

`OSError: [Errno 98] Address already in use`

Another process is using port 8080 — stop it or change `PORT` in the Python script and `VTN_PORT` in the sketch

curl upload returns `connection refused`

Opta not yet running the upload sketch, or wrong IP in the curl command